Course Name: Marketing Analytics

Course Code: DS 4115

Lab Manual 2: Exploratory Data Analysis (EDA) & Digital Conversion Funnel Analysis using Python

Lab Overview

In this lab, you will transition from spreadsheet analytics to a programmatic data science workflow. Working in **Google Colab** or **Jupyter Notebook**, you will ingest digital telemetry data, construct a multi-step conversion funnel (session_start →\rightarrow→ add_to_cart →\rightarrow→ purchase), handle missing values and data hygiene issues using pandas, and uncover digital conversion blockers using seaborn correlation heatmaps.

Learning Objectives

1. Write Python scripts to dynamically generate or load digital telemetry session logs.
2. Construct and analyze multi-step digital conversion funnels and drop-off rates.
3. Perform data cleaning and missing value imputation (pandas).
4. Execute Exploratory Data Analysis (EDA) to isolate user drop-off points.
5. Generate and interpret correlation heatmaps (seaborn) to identify key conversion drivers.

Part 0: Step-by-Step Dataset Acquisition & Setup Guide

Why Python for Digital Telemetry?

While Excel is great for tabular data, digital behavioral analytics (GA4 event logs, user clickstream telemetry) requires handling thousands of unstructured or semi-structured rows. In this lab, we will generate a synthetic GA4-style session log dataset directly inside Python to simulate real-world web and mobile app interactions.

Step 1: Open Google Colab

1. Navigate to Google Colab.
2. Create a **New Notebook** and rename it to Lab2_Conversion_Funnel_EDA.ipynb.

Step 2: Set Up Your Python Environment

Run the following code block in your first notebook cell to import the required data science libraries:

Python

```
# Import necessary libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
import seaborn as sns

# Set visualization styles
sns.set_theme(style="whitegrid")
print("Environment successfully set up!")
```

Part 1: Generating and Loading Digital Session Logs

Step 1: Generate Synthetic GA4 Session Logs

Copy and run the script below to generate 5,000 realistic digital session logs featuring user IDs, device categories, funnel progression flags, page views, and time spent on site:

Python

```
# Set random seed for reproducibility
np.random.seed(42)
n_rows = 5000

# Generate dummy session attributes
session_ids = [f"sess_{i:04d}" for i in range(1, n_rows + 1)]
user_ids = [f"user_{np.random.randint(1000, 2000)}" for _ in range(n_rows)]
device_categories = np.random.choice(['desktop', 'mobile', 'tablet'], size=n_rows, p=[0.5, 0.4, 0.1])

# Funnel flags (session_start is always 1, others follow realistic probabilities)
session_start_flag = np.ones(n_rows, dtype=int)
add_to_cart_flag = np.random.choice([0, 1], size=n_rows, p=[0.7, 0.3]) # 30% add to cart
purchase_flag = np.where(add_to_cart_flag == 1, np.random.choice([0, 1], size=n_rows, p=[0.4, 0.6]), 0) # 60% conversion from cart

page_views = np.random.poisson(lam=3, size=n_rows) + 1
time_on_site_sec = page_views * np.random.randint(20, 60, size=n_rows)

# Create DataFrame
df = pd.DataFrame({
    'session_id': session_ids,
    'user_id': user_ids,
    'device_category': device_categories,
    'session_start_flag': session_start_flag,
    'add_to_cart_flag': add_to_cart_flag,
    'purchase_flag': purchase_flag,
    'page_views': page_views,
    'time_on_site_sec': time_on_site_sec
})

# Introduce artificial missing values in 'time_on_site_sec' for data cleaning practice
df.loc[np.random.choice(n_rows, 50), 'time_on_site_sec'] = np.nan

# Save to CSV and inspect the first 5 rows
```

```
df.to_csv('ga4_sample_session_logs.csv', index=False)
print("Dataset 'ga4_sample_session_logs.csv' successfully generated and loaded!")
df.head()
```

Step 2: Inspect Dataset Structure

Run the following code to check column data types and non-null counts:

Python
```
df.info()
```

Part 2: Constructing the Digital Conversion Funnel

Step 1: Calculate Funnel Step Volumes

A conversion funnel tracks user progression through key milestones (session_start →\rightarrow→ add_to_cart →\rightarrow→ purchase). Run the script below to compute absolute user sessions and conversion rates relative to Step 1:

Python
```
# Calculate funnel metrics
total_sessions = df['session_id'].nunique()
sessions_with_cart = df[df['add_to_cart_flag'] == 1]['session_id'].nunique()
sessions_with_purchase = df[df['purchase_flag'] == 1]['session_id'].nunique()

# Create a summary DataFrame
funnel_data = {
    'Funnel Step': ['1. Session Start', '2. Add to Cart', '3. Purchase'],
    'User Sessions': [total_sessions, sessions_with_cart, sessions_with_purchase]
}
funnel_df = pd.DataFrame(funnel_data)

# Calculate conversion rates relative to step 1
funnel_df['Conversion Rate (%)'] = (funnel_df['User Sessions'] / total_sessions) * 100
print(funnel_df)
```

Part 3: Data Cleaning & Handling Missing Values

Real-world telemetry data is often messy, containing missing or corrupt records.

Step 1: Detect Missing Values

Check where missing data resides across columns:

Python
```
print(df.isnull().sum())
```

Step 2: Impute and Clean Data

Apply median imputation for numeric anomalies and clean string whitespace:

Python
```
# Impute missing time_on_site_sec with median
median_time = df['time_on_site_sec'].median()
df['time_on_site_sec'] = df['time_on_site_sec'].fillna(median_time)

# Drop rows with missing session IDs if any
df = df.dropna(subset=['session_id'])

# Clean categorical formatting
df['device_category'] = df['device_category'].str.lower().str.strip()
print("Data cleaning completed. Missing values remaining:", df.isnull().sum().sum())
```

Part 4: Exploratory Data Analysis & Correlation Heatmap

To isolate digital conversion blockers (e.g., identifying why users abandon sessions or fail to purchase), we will calculate statistical correlations across user engagement variables.

Step 1: Generate Correlation Matrix

Python
```
# Select numerical columns relevant to conversion behavior
numeric_cols = ['session_start_flag', 'add_to_cart_flag', 'purchase_flag', 'page_views', 'time_on_site_sec']
corr_matrix = df[numeric_cols].corr()
print(corr_matrix)
```

Step 2: Plot the Heatmap

Visualize the correlation matrix using seaborn:

Python
```
# Plot the correlation heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt=".2f", linewidths=0.5)
plt.title("Digital Behavioral Correlation Heatmap – Identifying Conversion Blockers", fontsize=12)
plt.show()
```

Deliverables Checklist for Students

- [ ] Successfully created and executed the Python script in Google Colab / Jupyter Notebook to generate ga4_sample_session_logs.csv.
- [ ] Built the 3-step digital conversion funnel table and computed step-by-step conversion percentages.
- [ ] Performed data cleaning, handled missing values via median imputation, and cleaned categorical fields using pandas.
- [ ] Generated and interpreted the seaborn correlation heatmap linking page views and session time to purchase conversion.
- [ ] Pushed the completed .ipynb notebook file to your individual student GitHub repository.